



CYBERDESK OS

Eine Cyberpunk-Desktop-Simulation in PySide6 — Kursprojekt GUI-Weiterbildung

• DAS WERKZEUG DAHINTER

PySide6 — die Python-Anbindung an Qt6

PySide6 ist die offizielle Python-Anbindung („Bindings“) an das C++-Framework Qt6 — bekannt als **Qt for Python**. Damit lassen sich plattformunabhängige Desktop-Anwendungen mit grafischer Oberfläche komplett in Python entwickeln, ohne eine Zeile C++ zu schreiben.

Offizieller Status

Wird direkt von The Qt Company entwickelt und gepflegt — kein Community-Fork.

Lizenz: LGPLv3

Kostenlose Nutzung auch in kommerziellen Projekten, ohne den eigenen Quellcode offenlegen zu müssen.

Kernmodule

QtWidgets (Buttons, Fenster), QtGui (Grafik, Schriften, Bilder), QtCore (Signals & Slots, Threads, Dateien).

Mitgelieferte Werkzeuge

pyside6-designer (visueller UI-Editor), pyside6-uic (wandelt Designs in Python-Code um).

• Worum geht es

Kein Betriebssystem. Eine Simulation eines Desktops.

Login, ein Desktop mit beweglichen Icons, ein echtes Fenstersystem, eine Taskbar mit Startmenü — und mehrere Apps, die jeweils in ihrem eigenen Fenster laufen. Alles eingebettet in eine durchgängige Neon-Ästhetik.

- ◆ **Funktional** — echtes Fenstersystem, mehrere Apps parallel offen
- ◆ **Atmosphärisch** — Cyberpunk-Look per zentralem Qt-Stylesheet
- ◆ **Erweiterbar** — neue Apps über eine gemeinsame Basisklasse
- ◆ **Ehrlich** — was nicht fertig wurde, steht offen in dieser Präsentation

• DAS HERZ DER APP

Kein Modul kennt ein anderes direkt

Alles läuft über einen globalen **EventBus** (Publish/Subscribe). Ein Klick auf ein Desktop-Icon publiziert `app.launch` — die **AppRegistry** reagiert, egal welche App das ist.



- DESIGN-ENTSCHEIDUNG

Das Fensterverhalten ist komplett selbst gebaut

PySide6 liefert das Fenster — aber sobald man mit `Qt.FramelessWindowHint` die native Windows-Dekoration abschaltet, verschwindet auch das komplette Fensterverhalten mit: Verschieben, Rand-Resize, Minimieren/Maximieren/Schließen. Genau das ist in `apps/base_app.py` selbst nachgebaut, um exakt der Figma-Vorlage zu entsprechen.

Eigene Titelleiste

App-Icon, Minimieren/Maximieren/Schließen als eigene, gestylte Buttons statt OS-Standard.

Eigener Rand-Resize

Fenstergröße per Maus an der Kante ändern — selbst implementiert, kein natives Verhalten.

Ein Preis dafür

Kein OS-Snapping, kein natives Alt+Tab-Icon — bewusst in Kauf genommen für den Look.

• WAS TATSÄCHLICH LAUFT

5 Apps, jede in ihrem eigenen Fenster



Taschenrechner

Standard +
wissenschaftlicher Modus

fertig



NetRunner

Echter Browser (Chromium)

fertig



Notizen

Mehrere Notizen, Auto-Save
30s

fertig



Datei-Explorer

Echtes VirtualFileSystem

teilweise



Einstellungen

Profil + Wallpaper

fertig

• EHRLICHER SCHLUSSTAND

Was fertig ist, was fehlt

● FERTIG

Kernsystem — EventBus, WindowManager, AppRegistry, BaseApp

5 Apps — Taschenrechner, NetRunner, Notizen, Datei-Explorer, Einstellungen

Desktop & Taskbar — Wallpaper, Kontextmenü, Startmenü, Minimieren/Wiederherstellen

13/13 Tests grün, ruff sauber

- TEILWEISE

Datei-Explorer — echte Navigation, aber kein Öffnen-in-App, kein Kontextmenü

Look & Feel — Farben/Glow ja, Scanlines/Sounds nein

Persistenz — Notizen/Profil ja, Fensterpositionen nein

● OFFEN

Terminal — nicht neu gebaut

Easter Eggs — keins umgesetzt

Drag & Drop zwischen Apps — nicht umgesetzt

● DANACH

Nächste Schritte